

YOWYOB ERP

Comptabilité Générale et Analytique

Spécification Backend

Mode Offline-First

Spécification backend, architecture offline-first et idempotence

Version	1.2
Date	Juillet 2026
Révision	Alignement handlers frontend CG/CA
Public	Équipe backend, architectes, chefs de projet
Statut	Document de référence contractuel

Le frontend gère IndexedDB, outbox et détection réseau. Le backend accepte la synchronisation différée.

Contents

1	Introduction	3
1.1	Objectif	3
1.2	Principe fondamental	3
2	Architecture offline-first	4
2.1	Vue d'ensemble	4
2.2	Les cinq couches	5
2.3	Patterns architecturaux	5
2.3.1	Pattern Outbox (file d'attente)	5
2.3.2	Pattern Repository (façade métier)	5
2.3.3	Pattern Online-first	6
2.3.4	Pattern Optimistic UI	6
2.4	Répartition des responsabilités	6
2.5	Flux de lecture	6
2.6	Flux d'écriture	6
2.7	Organisation backend recommandée	7
2.8	Handlers par entité (contrat frontend → backend)	7
2.9	Phases d'implémentation	7
2.10	Anti-patterns à éviter	8
3	L'idempotence — explication détaillée	9
3.1	Définition	9
3.2	Pourquoi c'est indispensable en offline-first	9
3.3	Mécanisme recommandé	10
3.3.1	En-tête Idempotency-Key	10
3.3.2	Corps avec clientId	10
3.3.3	Comportement serveur attendu	10
3.3.4	Stockage côté serveur	10
3.4	Exemples concrets	11
3.4.1	Création d'écriture comptable	11
3.4.2	Validation d'écriture	11
3.5	Ce qui n'est PAS de l'idempotence	11
4	Obligations transversales	12
4.1	Format de réponse	12
4.2	En-têtes obligatoires	12
4.3	Champs obligatoires sur chaque entité synchronisable	12
4.4	Codes d'erreur structurés	13
4.5	Health check	13

5	Sync incrémentale	14
5.1	Paramètre since	14
5.2	Endpoint batch (optionnel P2)	14
6	Comptabilité générale (CG)	15
6.1	État actuel	15
6.2	Écritures comptables — P0	15
6.3	Autres entités CG — P1	15
7	Comptabilité analytique (CA)	17
7.1	APIs à créer — P0	17
7.1.1	Écritures analytiques	17
7.1.2	Journaux analytiques — P0	17
7.1.3	Configuration — P1	17
7.1.4	Import CG vers CA — P1	17
8	Pièces jointes et audit	18
8.1	Pièces jointes — P2	18
8.2	Audit — P1	18
9	Ce que le backend ne doit pas faire	19
10	Priorisation	20
10.1	P0 — Minimum viable	20
10.2	P1 — Production fiable	20
10.3	P2 — Optimisation	20
11	Résumé	21

Chapter 1

Introduction

1.1 Objectif

Ce document liste **uniquement** les exigences côté backend pour un fonctionnement offline-first complet de l'ERP Yowyob.

1.2 Principe fondamental

Le backend n'est jamais "offline". Il reçoit les requêtes HTTP quand elles arrivent, y compris avec retard après une coupure réseau côté client.

Le frontend applique la règle suivante :

- **En ligne** : communication directe avec l'API (source de vérité serveur).
- **Hors ligne** : stockage local (IndexedDB) + file d'attente (outbox).
- **Retour en ligne** : rejeu automatique de la file vers le backend.

1.3 État d'alignement frontend ↔ backend (juillet 2026)

Le frontend a implémenté IndexedDB, outbox, cache lecture et handlers de sync. Le tableau ci-dessous résume l'**écart actuel** avec les exigences backend de ce document :

Composant	État frontend (implémenté)
Outbox + cache lecture CG	Écritures, journaux, plan comptable, exercices, périodes, budgets, opérations, taxes, devises, taux de change
Outbox notifications	Marquage POST .../notifications/{id}/read
Outbox CA (mock)	Centres, charges, comptes, plan analytique, journaux CA — en attente d'API
En-tête Idempotency-Key	Non envoyé encore par le client (stocké localement comme <code>clientMutationId</code>)
Champ <code>clientId</code> dans le corps	Non envoyé — IDs locaux <code>local-* / ec-offline-*</code> retirés avant POST, mapping client→serveur en IndexedDB

Composant	État frontend (implémenté)
Gestion HTTP 409	Non gérée côté UI
Sync delta ?since=	Non consommée — les listes font un GET complet
Session UI hors ligne	JWT expiré accepté si snapshot local ; la sync API exige un token valide
Service Worker (PWA)	Shell + pré-cache routes (production) — sans impact backend

Conséquence pour le backend : les endpoints listés dans ce document doivent être rendus **idempotents** même si le client n'envoie pas encore toutes les clés — le rejeu de file est déjà actif côté frontend.

Chapter 2

Architecture offline-first

2.1 Vue d'ensemble

L'architecture retenue est **online-first avec repli offline**. Le backend reste la **source de vérité** lorsque le réseau est disponible. Le frontend assure la continuité de service hors ligne via un cache local et une file de synchronisation.

COUCHE PRÉSENTATION (Frontend)

Pages React - mise à jour optimiste, indicateurs discrets

COUCHE DÉTECTION RÉSEAU (Frontend)

navigator.onLine + échecs API consécutifs

EN LIGNE

HORS LIGNE

API REST Backend
(source de vérité)

Cache IndexedDB
(lecture locale)

Outbox (file)
mutations

Moteur de sync
(retour en ligne)

Handlers par
entité métier

API REST Backend
+ idempotence

2.2 Les cinq couches

#	Couche	Rôle	Responsable
1	Présentation	Affichage immédiat, pas de blocage UI	Frontend
2	Détection réseau	Décision online / offline	Frontend
3	Cache lecture	Consultation hors ligne des listes	Frontend (IndexedDB)
4	Outbox	File d'attente des écritures	Frontend (IndexedDB)
5	Sync + API	Persistance définitive, règles métier	Backend

2.3 Patterns architecturaux

2.3.1 Pattern Outbox (file d'attente)

Toute mutation effectuée hors ligne est sérialisée dans une outbox persistante :

Action → IndexedDB → outbox (status: pending)
→ [retour en ligne] → API Backend → outbox (status: done)

Exigence backend : accepter le rejeu idempotent de ces opérations (voir chapitre Idempotence).

2.3.2 Pattern Repository (façade métier)

Le frontend n'appelle pas l'API directement depuis les pages. Une couche intermédiaire décide :

if (en ligne) → appel API backend
if (hors ligne) → cache + outbox

Exigence backend : API stables, format de réponse uniforme.

2.3.3 Pattern Online-first

Tant que le réseau et l'API répondent, **toute lecture et écriture** passe par le backend. Le cache local est un **repli**, pas la source principale.

2.3.4 Pattern Optimistic UI

L'interface se met à jour avant la confirmation serveur. Le backend valide ensuite et peut rejeter (422) ou signaler un conflit (409).

2.4 Répartition des responsabilités

Frontend	Backend
IndexedDB, outbox, détection réseau	Persistance définitive en base
Cache des listes pour lecture offline	Endpoints REST CRUD
Rejeu de la file à la reconnexion	Idempotence (Idempotency-Key)
Mapping ID client → ID serveur	Acceptation de clientId à la création
Indicateurs UI (sync en attente)	Versionnement (version / updatedAt)
Service Worker (shell PWA, optionnel)	Gestion des conflits (HTTP 409)
Pause du polling AJAX hors ligne	Règles métier comptables (validation, clôture)
Sync delta (consommation)	Sync delta (fourniture ?since=)

2.5 Flux de lecture

1. **En ligne** : GET API → mise à jour cache local → affichage.
2. **Hors ligne** : lecture cache IndexedDB → affichage.
3. **Échec réseau** : repli sur le cache sans bloquer l'utilisateur.

Backend : fournir GET /ressource?since=timestamp pour optimiser la resynchronisation.

2.6 Flux d'écriture

1. **En ligne** : POST/PUT/DELETE direct → réponse serveur → mise à jour UI.
2. **Hors ligne** : sauvegarde locale + ajout outbox → UI optimiste.
3. **Retour en ligne** : flush outbox dans l'ordre chronologique → API backend.

4. **Succès** : statut outbox = **done** (uniquement après accusé serveur).
5. **Conflit** : HTTP 409 → résolution manuelle côté UI.

Backend : idempotence + versionnement sur chaque mutation.

2.7 Organisation backend recommandée

Controller REST

↓

Service métier (règles comptables)

↓

Repository / DAO

↓

Base de données

Couche transversale :

- Filtre Idempotency-Key (avant controller)
- Gestion version / If-Match (sur PUT)
- Table idempotency_keys (TTL 24-72h)
- Journal d'audit sync

2.8 Handlers par entité (contrat frontend → backend)

Le frontend enregistre un handler de sync par type d'entité (`lib/offline/sync-engine.ts`).

Le backend doit exposer les endpoints correspondants :

Entité outbox	Endpoints backend	Statut
écriture_comptable	/api/accounting/ecritures (+ validate, deactivate)	Existe — à étendre (P0)
cg.journaux	/api/accounting/journals	Existe — à étendre (P0 bis)
cg.plan_comptable	/api/accounting/plan-comptable	Existe — à étendre (P0 bis)
cg.exercices	/api/accounting/exercices (+ close)	Existe — à étendre (P0 bis)
cg.periodes	/api/accounting/periodes (+ close)	Existe — à étendre (P0 bis)
cg.budgets	/api/accounting/budgets (+ validate, activate)	Existe — à étendre (P0 bis)
cg.operations	/api/accounting/operations	Existe — à étendre (P0 bis)
cg.taxes	/api/accounting/taxes	Existe — à étendre (P0 bis)
cg.devises	/api/accounting/currencies	Existe — à étendre (P0 bis)

Entité outbox	Endpoints backend	Statut
cg.devises_rates	/api/accounting/exchange-rates (POST taux)	Existe — cas particulier
notifications	/api/accounting/notifications/{id}/thead	Idempotent POST
ecriture_analytique	/api/accounting/analytic-entries	À créer (P0)
ca.centres	/api/accounting/analytic-centres (proposé)	À créer (P0 CA)
ca.charges	/api/accounting/analytic-charges (proposé)	À créer (P0 CA)
ca.comptes	/api/accounting/analytic-accounts (proposé)	À créer (P0 CA)
ca.plan_comptes	/api/accounting/analytic-accounts/plan (proposé)	À créer (P1 CA)
ca.journaux	/api/accounting/analytic-journals	À créer (P0)

Note — taux de change (cg.devises_rates) : le frontend ne crée pas une ligne de liste dédiée ; il envoie un POST `/api/accounting/exchange-rates` avec `devise_source_id`, `devise_cible_id` et `taux`. La clé outbox est composite (`sourceId:targetId`).

2.8.1 Double mécanisme d'identification client (recommandé)

Le backend doit accepter **l'un ou l'autre** (idéalement les deux) :

1. **Idempotency-Key** = `clientMutationId` de l'outbox (en-tête HTTP).
2. **clientId** explicite dans le corps JSON à la création.
3. **Déduplication par préfixe** : si un enregistrement avec le même `clientId` existe déjà pour l'organisation, retourner 200 `ALREADY_PROCESSED`.

Le frontend actuel utilise surtout le mécanisme (3) implicitement via suppression de l'id local ; le mécanisme (1) est la priorité d'implémentation conjointe frontend/backend. Voir aussi le tableau d'alignement au chapitre 1.

2.9 Phases d'implémentation

Phase	Contenu
1	Fondations : idempotence, clientId, format réponse, health check
2	Sync delta : paramètre ?since= sur les listes
3	APIs comptabilité analytique complètes
4	Endpoint batch <code>/api/accounting/sync/push</code> (optionnel)

2.10 Anti-patterns à éviter

Anti-pattern	Conséquence
Pas d'idempotence	Doublons à la reconnexion
Écrasement silencieux (pas de 409)	Perte de données
Session “offline” côté serveur	Complexité inutile
Validation uniquement côté client	Écritures invalides en base

Chapter 3

L’idempotence — explication détaillée

3.1 Définition

Une opération est **idempotente** si l’exécuter **une fois** ou **plusieurs fois** produit le **même résultat final**, sans effet de bord supplémentaire.

En pratique pour une API REST :

Si le client envoie deux fois la même requête de création (à cause d’un timeout, d’une reconnexion ou d’un jeu de file d’attente), le serveur ne doit **pas créer deux enregistrements identiques**.

3.2 Pourquoi c’est indispensable en offline-first

Lorsqu’un utilisateur travaille hors ligne, le frontend enregistre chaque action dans une **outbox** (file d’attente). Au retour en ligne, le moteur de synchronisation **rejoue** ces opérations.

Scénario sans idempotence :

1. L’utilisateur crée une écriture hors ligne.
2. La connexion revient ; le frontend envoie `POST /ecritures`.
3. Le serveur reçoit la requête, crée l’écriture, mais la réponse HTTP est perdue (timeout).
4. Le frontend considère l’échec et **rejoue** la même requête.
5. Sans idempotence : **deux écritures identiques** en base.

Scénario avec idempotence :

1. Même contexte : le client renvoie la requête avec la même `Idempotency-Key`.
2. Le serveur reconnaît la clé déjà traitée.
3. Il retourne `200 OK` avec l’entité existante (pas de doublon).

3.3 Mécanisme recommandé

3.3.1 En-tête Idempotency-Key

Chaque mutation (CREATE, UPDATE critique, VALIDATE) doit accepter :

POST /api/accounting/ecritures

Headers :

```
Authorization: Bearer {token}
X-Organization-Id: {orgId}
X-Tenant-Id: {tenantId}
Idempotency-Key: cm-uuid-unique-par-operation-client
```

3.3.2 Corps avec clientId

En complément, le corps peut porter un identifiant stable généré côté client :

```
{
  "clientId": "ec-offline-20260708-abc123",
  "libelle": "Facture fournisseur",
  "dateEcriture": "2026-07-08",
  ...
}
```

État frontend (v1.2) : le champ `clientId` n'est pas encore envoyé dans le corps. Le client génère un ID local (`local-*` ou `ec-offline-*`), le retire du corps au POST, puis enregistre le mapping vers l'UUID serveur après 201 Created. Le backend doit néanmoins accepter `clientId` dès que le frontend l'enverra, et peut d'ores et déjà déduplicer via Idempotency-Key.

3.3.3 Comportement serveur attendu

Situation	Réponse attendue
Première requête avec une clé inconnue	201 Created + entité créée
Rejeu avec la <i>même</i> Idempotency-Key	200 OK + <i>même</i> entité, message ALREADY_PROCESSED
Requête avec une nouvelle clé mais même <code>clientId</code> déjà en base	200 OK + entité existante (déduplication)
Clé expirée (ex. > 24 h)	Traiter comme nouvelle requête ou 409 selon politique

3.3.4 Stockage côté serveur

Le backend doit conserver temporairement (Redis, table dédiée) :

- `idempotency_key`

- `organization_id`
- `http_status` de la réponse originale
- `response_body` (ou référence entité créée)
- `expires_at` (TTL recommandé : 24 à 72 heures)

3.4 Exemples concrets

3.4.1 Création d'écriture comptable

Requête 1 (première sync) :

```
POST /api/accounting/ecritures
Idempotency-Key: save-ec-offline-001
```

→ 201 Created

```
{ "success": true, "data": { "id": "srv-uuid-99", "clientId": "ec-offline-001" } }
```

Requête 2 (rejeu accidentel) :

```
POST /api/accounting/ecritures
Idempotency-Key: save-ec-offline-001
```

→ 200 OK

```
{ "success": true, "message": "ALREADY_PROCESSED",
  "data": { "id": "srv-uuid-99", "clientId": "ec-offline-001" } }
```

3.4.2 Validation d'écriture

POST /ecritures/{id}/validate avec Idempotency-Key: validate-ec-001 :

- 1ère fois : écriture validée, 200 OK.
- 2ème fois : écriture déjà validée, 200 OK sans erreur (pas de double validation métier).

3.5 Ce qui n'est PAS de l'idempotence

Cas	Explication
Deux factures différentes avec deux clés différentes	Normal : deux créations distinctes
Modification (PUT) avec version obsolète	Conflit 409, pas idempotence — voir chapitre Conflits
DELETE puis CREATE même clientId	Politique métier à définir (généralement interdit)

Chapter 4

Obligations transversales

4.1 Format de réponse

```
{
  "success": true,
  "message": "...",
  "data": { ... },
  "timestamp": "2026-07-08T12:00:00Z",
  "code": 200
}
```

4.2 En-têtes obligatoires

En-tête	Rôle
Authorization	Token JWT
X-Organization-Id	Organisation propriétaire des données
X-Tenant-Id	Tenant plateforme
Idempotency-Key	Idempotence des mutations (voir chapitre 2)
If-Match	Version attendue sur PUT/PATCH (optionnel mais recommandé)

4.3 Champs obligatoires sur chaque entité synchronisable

Champ	Type	Usage
id	UUID serveur	Référence stable
clientId	UUID (optionnel en entrée)	ID généré côté client hors ligne
version	entier	Détection conflits (incrémenté à chaque modification)
createdAt	ISO 8601	Ordre chronologique
updatedAt	ISO 8601	Sync delta (?since=)

4.4 Codes d'erreur structurés

HTTP	Code métier	Cas
422	VALIDATION_ERROR	Règle comptable (équilibre, période clôturée)
409	VERSION_CONFLICT	Modification concurrente
403	FORBIDDEN	Droits insuffisants
404	NOT_FOUND	Entité supprimée côté serveur

Réponse conflit 409 :

```
{
  "success": false,
  "code": "VERSION_CONFLICT",
  "message": "L'entité a été modifiée par un autre utilisateur",
  "data": {
    "serverEntity": { ... },
    "clientEntity": { ... }
  }
}
```

4.5 Health check

```
GET /api/health
→ { "status": "UP" }
```


Chapter 5

Sync incrémentale

5.1 Paramètre since

Sur chaque liste :

```
GET /api/accounting/ecritures?since=2026-07-08T10:00:00Z
```

Réponse :

```
{
  "success": true,
  "data": {
    "items": [ ... modifications depuis since ... ],
    "deletedIds": ["uuid-supprime-1"],
    "syncToken": "2026-07-08T12:00:00Z"
  }
}
```

5.2 Endpoint batch (optionnel P2)

```
POST /api/accounting/sync/push
```

Body: {

```
  "operations": [
    { "clientMutationId": "...", "entity": "ecriture_comptable",
      "action": "CREATE", "payload": { ... } }
  ]
}
```

}

```
→ { "results": [ { "clientMutationId", "status", "serverId", "error" } ] }
```

Chapter 6

Comptabilité générale (CG)

6.1 État actuel

Les endpoints CG existent (écritures, plan comptable, journaux, exercices, périodes, budgets, opérations, taxes, devises, taux de change). Il faut les **étendre** (idempotence, clientId, version), pas les recréer.

6.2 Écritures comptables — P0

Méthode	Endpoint	Extension backend requise
POST	/api/accounting/ecritures	clientId + Idempotency-Key
PUT	/api/accounting/ecritures/{id}	version + 409 si conflit
POST	/api/accounting/ecritures/{id}/validate	Idempotent
DELETE	/api/accounting/ecritures/{id}	Idempotent (200 si déjà supprimé)
PATCH	/api/accounting/ecritures/{id}/deactivate	Idempotent
GET	/api/accounting/ecritures	Paramètre ?since=
GET	/api/accounting/ecritures/non-validated	Paramètre ?since=

Règles métier serveur obligatoires :

- Équilibre débit/crédit
- Période non clôturée
- Contrôle des droits par rôle
- Numérotation des pièces (serveur = source de vérité)
- Validation irréversible une fois confirmée

6.3 Autres entités CG — P0 bis / P1

Les endpoints suivants **existent déjà**. Il faut y appliquer le même contrat (clientId, version, ?since=, idempotence) sans les recréer :

Entité	Endpoint	Priorité
Plan comptable	/api/accounting/plan-comptable	P0 bis
Journaux	/api/accounting/journals	P0 bis
Exercices	/api/accounting/exercices	P0 bis
Périodes	/api/accounting/periodes	P0 bis
Budgets	/api/accounting/budgets	P0 bis
Opérations comptables	/api/accounting/operations	P0 bis
Taxes	/api/accounting/taxes	P0 bis
Devises	/api/accounting/currencies	P0 bis
Taux de change	/api/accounting/exchange-rates	P0 bis
Axes analytiques (lecture CG)	/api/accounting/analytics	P1
Notifications (marquer lu)	/api/accounting/notifications/{id}/read	P0

Cas particulier — taux de change : idempotence sur le couple (devise_source_id, devise_cible_id, date_effet) ou sur Idempotency-Key. Un rejeu ne doit pas créer deux taux actifs contradictoires.

6.4 Paramètres et notifications — P1

Méthode	Endpoint
POST	/api/accounting/notifications/{id}/read (idempotent)
GET	/api/accounting/notifications/unread
GET	/api/accounting/notifications/stream (SSE, existant)

Chapter 7

Comptabilité analytique (CA)

7.1 État actuel et périmètre

La comptabilité analytique repose encore en partie sur un **stockage local frontend** (IndexedDB + outbox). Les entités **ca.*** sont en file d'attente mais le handler sync est un no-op tant que les APIs ci-dessous n'existent pas. Pour un offline-first complet, le backend doit exposer les ressources correspondantes.

7.2 APIs à créer — P0

7.2.1 Écritures analytiques

Méthode	Endpoint
GET	/api/accounting/analytic-entries
GET	/api/accounting/analytic-entries/{id}
GET	/api/accounting/analytic-entries?since=
GET	/api/accounting/analytic-entries?statut=BROUILLON
POST	/api/accounting/analytic-entries
PUT	/api/accounting/analytic-entries/{id}
DELETE	/api/accounting/analytic-entries/{id}
POST	/api/accounting/analytic-entries/{id}/validate
POST	/api/accounting/analytic-entries/{id}/reject

7.2.2 Journaux analytiques — P0

CRUD sur `/api/accounting/analytic-journals` (entité outbox `ca.journaux`).

7.2.3 Centres, charges et comptes analytiques — P0 CA

Le frontend synchronise déjà ces entités en outbox (mode mock). Endpoints proposés :

Entité outbox	Méthode	Endpoint proposé
ca.centres	CRUD	/api/accounting/analytic-centres
ca.charges	CRUD	/api/accounting/analytic-charges
ca.comptes	CRUD	/api/accounting/analytic-accounts
ca.plan_comptes	CRUD	/api/accounting/analytic-accounts (plan / nomenclature)

Même contrat que la CG : `clientId`, `Idempotency-Key`, `version`, `?since=`, réponses 409 en conflit.

7.2.4 Configuration — P1

GET/PUT /api/accounting/analytic-config
 → { "importComptabiliteGeneraleActive": true }

7.2.5 Import CG vers CA — P1

POST /api/accounting/analytic-entries/import-from-general
 → { "created": [...], "ignored": 3 }

Chapter 8

Pièces jointes et audit

8.1 Pièces jointes — P2

POST /api/accounting/attachments
Idempotency-Key + multipart
→ { "id", "clientId", "url" }

8.2 Audit — P1

Chaque opération synchronisée doit être tracée :

- `clientMutationId`
- Utilisateur
- Horodatage
- Action (CREATE, UPDATE, VALIDATE, etc.)

Chapter 9

Ce que le backend ne doit pas faire

À éviter	Raison
Mode offline côté serveur	Le offline est géré par le client
WebSocket obligatoire	Sync à la reconnexion suffit
Écrasement silencieux en conflit	Perte de données
Doublons sur rejeu de requête	Sans idempotence

Chapter 10

Priorisation

10.1 P0 — Minimum viable

1. Idempotency-Key + clientId sur POST écritures CG
2. version / updatedAt + 409 sur PUT écritures CG
3. APIs écritures analytiques complètes
4. Format de réponse stable
5. GET /api/health

10.2 P0 bis — Déjà consommé offline (endpoints existants)

1. Idempotence sur : journaux, plan comptable, exercices, périodes, budgets
2. Idempotence sur : opérations comptables, taxes, devises
3. Idempotence sur POST taux de change (/api/accounting/exchange-rates)
4. Retour systématique de l'id serveur dans data après création (pour mapping client)

10.3 P1 — Production fiable

1. Paramètre ?since= sur les listes
2. APIs centres, charges, comptes analytiques (CA)
3. APIs journaux et config analytique
4. Import CG → CA côté serveur
5. Journal d'audit des sync
6. POST notifications/{id}/read idempotent

10.4 P2 — Optimisation

1. POST /api/accounting/sync/push (batch)
2. Snapshot dashboard
3. Upload pièces jointes idempotent

Chapter 11

Résumé

En une phrase : le backend doit accepter des écritures avec un ID client et une clé d'idempotence, versionner chaque entité, répondre 409 en cas de conflit, étendre l'idempotence aux entités CG déjà synchronisées par le frontend (journaux, taxes, devises, opérations, etc.), exposer les APIs analytiques manquantes (écritures, centres, charges, comptes), et permettre la sync delta via `?since=`.

Version 1.2 — juillet 2026 : alignement sur l'implémentation frontend réelle (out-box multi-entités CG, CA mock, écarts Idempotency-Key / clientId documentés).

L'**idempotence** garantit qu'une coupure réseau ou un rejeu de la file d'attente ne crée jamais de doublons en base de données.